

Qudit-inspired optimization for graph coloring

David Jansen,¹ Timothy Heightman,¹ Luke Mortimer,¹ Ignacio Perito,¹ and Antonio Acín^{1,2}

¹*ICFO-Institut de Ciències Fotoniques, The Barcelona Institute of Science and Technology, 08860 Castelldefels (Barcelona), Spain*

²*ICREA-Institució Catalana de Recerca i Estudis Avançats, Lluís Companys 23, 08010 Barcelona, Spain*

(Dated: December 17, 2024)

We introduce a quantum-inspired algorithm for graph coloring problems (GCPs) that utilizes qudits in a product state, with each qudit representing a node in the graph and parameterized by d -dimensional spherical coordinates. We propose and benchmark two optimization strategies: qudit gradient descent, initiating qudits in random states and employing gradient descent to minimize a cost function, and qudit local quantum annealing, which adapts the local quantum annealing method to optimize an adiabatic transition from a tractable initial function to a problem-specific cost function. Our approaches are benchmarked against established solutions for standard GCPs, showing that our methods not only rival but frequently surpass the performance of recent state-of-the-art algorithms in terms of solution quality and computational efficiency. The adaptability of our algorithm and its high-quality solutions, achieved with minimal computational resources, point to an advancement in the field of quantum-inspired optimization, with potential applications extending to a broad spectrum of optimization problems.

I. INTRODUCTION

Today, many prominent tasks in industry can be formulated as integer optimization problems, such as how to efficiently use resources [1] or do tasks such as job scheduling [2] or portfolio optimization [3]. One important example of integer optimization is the graph coloring problem (GCP): given a graph with V nodes and $\mathcal{E}$ edges, assign colors to the nodes such that no two connected nodes share the same color. More generally, one might try to find the minimum number of colors needed for such an assignment to be possible, henceforth referred to as the *chromatic number* of a graph [4]. Being NP-complete [5], the GCP can be related to a large number of real-world problems. These include map coloring (e.g., assigning colors to provinces in Spain while avoiding that neighboring provinces have the same color), air traffic control [6], charging electric vehicles [7], resource allocation (e.g., supply chain or portfolio management), even tasks like designing seating plans, sports tournaments, or sudoku, and managing taxi bookings [4]. Thus, there is much active research into developing algorithms that efficiently find good or optimal solutions to the GCP and scale favorably with the size of the graph.

Quantum technologies might provide one avenue for efficiently solving optimization problems through algorithms such as the Quantum Approximate Optimization Algorithm (QAOA) [8–10] or quantum annealing [11–13]. While there is a straightforward mapping from binary optimization problems to qubit systems [14, 15], some integer optimization problems, like GCPs, are more naturally formulated using larger local degrees of freedom. One way to accomplish this in a physics-inspired manner is to extend the notion of a qubit to a d -dimensional local degree of freedom, known as a *qudit*. This, together with the improved capacity for simulations of physical systems with a larger number of local physical degrees of freedom, has inspired the development of hardware implementing

qudit systems and qudit-inspired algorithms [16–18]. For instance, in Ref. [7], Deller et al. extend QAOA and integer optimization problems, including the GCP, to the qudit regime, and Ref. [19] explores using qutrits (three-dimensional local degrees of freedom) for the GCP.

Complementary to the improvement of quantum technologies, the connection between quantum physics and optimization problems has enhanced the interest in so-called “physics-inspired algorithms” for optimization. A variety of such algorithms have emerged based on techniques like mean field theory [20–22], tensor networks [23, 24], neural networks [25–28], and dynamical evolution [29–31]. Some of these new approaches have enjoyed significant success, providing state-of-the-art solutions to existing problems as well as benchmarks to compare against possible quantum-hardware-based solutions.

In this work, we introduce a qudit-inspired algorithm for solving integer optimization problems. First, our approach maps the problem to a qudit system where each qudit is parametrized by d -dimensional spherical coordinates. Then, we propose two schemes to solve the optimization problem. In the first scheme, which we refer to as qudit gradient descent (QdGD), each qudit is initialized to a random state and we minimize a cost function via gradient descent. In the second scheme, which we refer to as qudit local quantum annealing (QdLQA), we simulate the quantum annealing from the ground state of a simple Hamiltonian to that of a problem-specific cost function. This is the concept of local quantum annealing introduced by Bowles et al. [22], in the sense that the system is constrained to remain in a product state through its evolution. We extend this idea to integer optimization, without requiring that the cost function at the end of the evolution has to have an interpretation as a quantum physical Hamiltonian. Our intuition for this approach is that we find the minima of a complicated

function $f_{\text{final}}(\vec{\phi})$ by first initializing the parameters, $\vec{\phi}$, to a known minimum of a simple function $f_{\text{initial}}(\vec{\phi})$. Then, by slowly changing $f_{\text{initial}}(\vec{\phi}) \rightarrow f_{\text{final}}(\vec{\phi})$ while minimizing with respect to the parameters $\vec{\phi}$, an optimal or good solution to $f_{\text{final}}(\vec{\phi})$ is found.

We test the algorithms on different standard GCPs and compare to different benchmarks in the literature [32–34]. In almost all cases, our approach obtains results as good as or better than recently proposed state-of-the-art algorithms such as graph-neural-network-based approaches [33, 34]. We also run our algorithms on large sparse datasets where many colors/local degrees of freedom (up to 70 [32]) are required to solve the problem. We find that even though our approaches do not find the global minimum for such high-dimensional problems, they still provide relatively good solutions (errors of the order 10^{-3}) for problem sizes unaccounted for in state-of-the-art works such as [33, 34]. Notably, in the case of QdLQA, we find that all of these results can be obtained doing only a few gradient descent steps (for most problems only one) at each point in the adiabatic evolution, as was achieved in Ref. [22].

The rest of this paper is structured as follows. In Sec. II, we introduce the GCP and the algorithms. In Sec. III we benchmark our approach with other state-of-the-art methods. In Sec. IV, we provide a summary and a discussion of improvements and further applications.

II. METHOD

A. Graph coloring

In our approach, we aim to solve the GCP by reframing it as finding the ground state of the antiferromagnetic Potts model [35, 36]. The two are closely related [33, 37, 38], and for an undirected graph $G = (V, \mathcal{E})$ with a set of nodes V , edges $\mathcal{E}$, and a set of c colors $\{0, \dots, c-1\}$, the energy is given by

$$E_{\text{Potts}} = \sum_{(i,j) \in \mathcal{E}} \delta(\sigma_i, \sigma_j), \quad (1)$$

where each spin is set to one of the colors $\sigma_i \in \{0, \dots, c-1\}$ and $\delta(\sigma_i, \sigma_j) = 1$ if $\sigma_i = \sigma_j$ and $\delta(\sigma_i, \sigma_j) = 0$ otherwise. Thus, this energy is minimized whenever as many spins as possible have a different color to each of their neighbors (so that as many terms as possible are zero). The chromatic number, $\chi(G)$, is defined as the minimum number of colors c required for this cost to reach zero (i.e., when no neighboring vertices share the same color). This problem can be posed in a number of forms, either to simply minimize the cost (NP-complete), to check whether a given graph admits a coloring with a certain number of colors (NP-complete), or to find the chromatic number of a graph (NP-hard) [5].

B. Qudit ansatz

To solve GCPs, we formulate them as optimization problems for qudits. We follow Ref. [7] and define the z angular momentum operator for site i , $\hat{L}_i^z$, and the eigenbasis $|l, m\rangle_i$, where $l = (c-1)/2$ and $m \in \{-l, \dots, l\}$ so that

$$\hat{L}_i^z |l, m\rangle_i = m |l, m\rangle_i. \quad (2)$$

We also introduce

$$\hat{L}_i^+ |l, m\rangle_i = \sqrt{(l-m)(l+m+1)} |l, m+1\rangle_i, \quad (3)$$

$$\hat{L}_i^- |l, m\rangle_i = \sqrt{(l+m)(l-m+1)} |l, m-1\rangle_i, \quad (4)$$

and the x angular momentum operator

$$\hat{L}_i^x = \frac{1}{2}(\hat{L}_i^+ + \hat{L}_i^-). \quad (5)$$

Furthermore, we parameterize each local state by c -dimensional spherical coordinates using $c-1$ angles

$$|\psi\rangle_i = \vec{\psi}_i = \begin{bmatrix} \cos(\phi_0^i) \\ \sin(\phi_0^i) \cos(\phi_1^i) \\ \sin(\phi_0^i) \sin(\phi_1^i) \cos(\phi_2^i) \\ \vdots \\ \sin(\phi_0^i) \sin(\phi_1^i) \dots \sin(\phi_{c-3}^i) \cos(\phi_{c-2}^i) \\ \sin(\phi_0^i) \sin(\phi_1^i) \dots \sin(\phi_{c-3}^i) \sin(\phi_{c-2}^i) \end{bmatrix}. \quad (6)$$

We choose spherical coordinates for the local states since they automatically keep the state normalized and are easily differentiable, however, other parametrizations are possible (such as using Cartesian coordinates).

Finally, we take a product-state ansatz for the total quantum state

$$|\psi\rangle = \prod_{i \in V} |\psi\rangle_i. \quad (7)$$

The QdLQA approach is inspired by local quantum annealing [22], and our goal is to start at the known minimum of one function and simulate a "quantum adiabatic evolution" into the minimum of a cost function for which the solution solves the optimization problem. During the quantum adiabatic evolution, we require that the system remains in a product state. This restriction prevents entanglement buildup in the system, which is believed to lead to the emergence of Barren plateaus [39, 40]. At the end of the simulation, each local state $\vec{\psi}_i$ is assigned a color, and Eq. (1) is used to calculate the energy. The scheme is illustrated in Fig. 1.

To simplify notation, we define the local probability vector $\vec{p}_i = \vec{\psi}_i^2$, where each component (denoted by m) has a clear physical interpretation, namely they give the probability of measuring the m -th eigenvalue of $\hat{L}_i^z$. This means that summing over all m components for node i

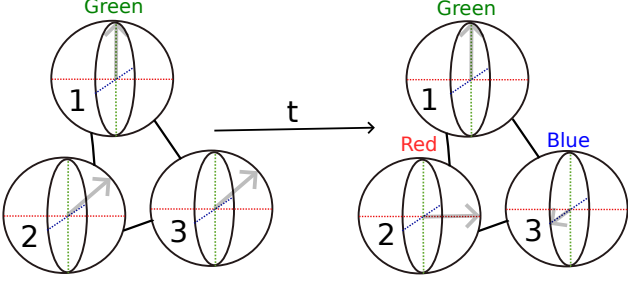


FIG. 1. Three nodes (labeled 1, 2, and 3) connected by three edges. We want to assign three colors (green, red, and blue) so that no two interacting nodes have the same color. The local state is represented by a vector in a three-dimensional sphere (gray arrow). In our initial state, we assign green to node 1, and nodes 2 and 3 are in the ground states of $-\hat{L}_2^x$ and $-\hat{L}_3^x$ respectively (drawn illustratively as a superposition of the three colors). As time t goes from 0 to 1, Eq. (11) is minimized with gradient descent. During the process, the spins corresponding to nodes 2 and 3 are assigned one eigenstate (color) of $\hat{L}_2^z$ and $\hat{L}_3^z$ (corresponding to the highest measurement probability) to provide a solution to Eq. (1).

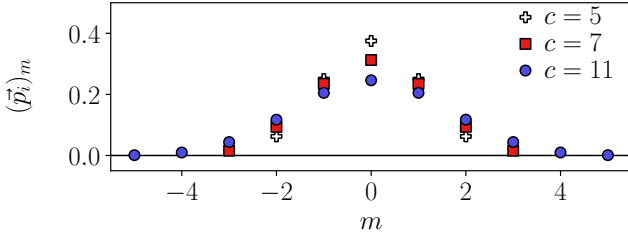


FIG. 2. The m components of the initial distribution of the ground state of $-\hat{L}_i^x$ in the $\hat{L}_i^z$ eigenbasis. Each of the components corresponds to a different eigenvalue of $\hat{L}_i^z$ [see Eq. (2)], or equivalently, a different color. We show the distribution for different numbers of colors c (local Hilbert space dimension).

gives $\sum_m (\vec{p}_i)_m = 1$. At each step in the quantum adiabatic evolution, a color configuration is generated by assigning each qudit i to the state (color) corresponding to maximum probability $\text{argmax}(\vec{p}_i)$.

We choose the initial cost function to be

$$E_I(\psi) = - \sum_{i \in V \setminus \{j_{\max}\}} \langle \hat{L}_i^x \rangle, \quad (8)$$

where $\psi := \psi(\vec{\phi})$ is the quantum state that depends on the angles $\vec{\phi} := (\phi_0^0, \phi_1^0, \dots, \phi_{c-2}^0, \phi_0^1, \dots, \phi_0^{|V|}, \dots, \phi_{c-2}^{|V|})$ for a system with c colors and $|V|$ nodes [see Eqs. (6) and (7)]. The parameters appear explicitly in the expression $\langle \hat{L}_i^x \rangle = {}_i \langle \psi | \hat{L}_i^x | \psi \rangle$, and each qudit is initialized to the ground state of $-\hat{L}_i^x$ (which is a superposition of different colors). The initial distribution is shown in Fig. 2. We note that the sum in Eq. (8) goes over all qudits except $j_{\max}$, which is the one with the most edges. This qudit is initialized with one color, e.g. $(1, 0, 0, \dots, 0)^T$, and is left

untouched during the adiabatic evolution. The ability to do this is due to the invariance of any solution when swapping the labels of the colors, as such, we are free to choose the color for one single spin (the one with the most edges to simplify as much as possible).

For our final cost function, we choose the scalar product between probability vectors connected by the edges. This choice arises naturally from our ansatz since two orthogonal $\vec{p}_i$ correspond to different colors. Using $\vec{p}_i$ instead gives the components the interpretation of a measurement probability and emits antiferromagnetic (antiparallel) solutions:

$$E_F(\psi) = \sum_{(i,j) \in \mathcal{E}} J_{i,j} \vec{p}_i^T \cdot \vec{p}_j, \quad (9)$$

where $J_{i,j} = 1 + h_{i,j}$, and $h_{i,j}$ are drawn from the uniform distribution in the interval $h_{i,j} \in [0, h)$ each time $E_F(\psi)$ is called. The reason for introducing the hyperparameter h is to help the simulation escape local minima. If the nodes i and j do not have a color clash, $\vec{p}_i^T \cdot \vec{p}_j$ will be approximately 0, and adding $h_{i,j}$ will not matter. However, for $\vec{p}_i^T \cdot \vec{p}_j > 0$, $h_{i,j}$ will increase the interaction strength between the nodes and encourage a change of colors. Furthermore, we note that a similar cost function (without the h hyperparameter) is used in graph neural network approaches [33, 34]. There, however, the $\vec{p}_i$ emerge from applying the softmax function to the final node embedding. Note that contrary to the algorithm of [22], this cost function does not have an interpretation as the expectation value of a physical Hamiltonian on the quantum product states considered. Alternatively, we could also have formulated a cost function corresponding to Eq. (1) as a physical Hamiltonian in terms of polynomials in $\hat{L}_i^z$ and $\hat{L}_j^z$ [7].

Furthermore, we follow Ref. [34] and introduce a term to force the qudits into superpositions of all the colors,

$$E_W(\psi) = \gamma \sum_{i \in V} \vec{p}_i^T \cdot \log(\vec{p}_i), \quad (10)$$

where $\gamma > 0$ is a hyperparameter and the logarithm is taken component wise. For large γ , this term favors a finite probability for all possible colors, even those that are initialized close to zero in the ground state of $-\hat{L}_i^x$ (see Fig. 2). Heuristically, we find that this term helps to avoid getting stuck in local minima (similar to what was observed with graph neural networks in Ref. [34]).

Finally, we conduct a quantum adiabatic evolution so that the total cost function becomes

$$E_{\text{Total}}(\psi, t) = (1 - t)E_I(\psi) + t(E_F(\psi) + E_W(\psi)). \quad (11)$$

In the simulation, we go from $t = 0$ to $t = 1$ using N_{steps} time steps, and at each point in time, we conduct α gradient descent steps. For some of the datasets discussed in Sec. III, we also find it beneficial not to start in the exact ground state of $-\hat{L}_i^x$, but rather to start in states where we perturb the initial angles in Eq. (6) to

$\phi_m^i \rightarrow \phi_m^i + \epsilon_m$, where ϵ_m are randomly chosen from the uniform distributed in the interval $[-f, f)$, and f is a hyperparameter. Furthermore, we generate the classical color configuration at each time step and store the one with the lowest value for E_{Potts} [see Eq. (1)], which provides the best configuration found by the algorithm.

For the alternative method focusing entirely on gradient descent, QdGD, each qudit is initialized in a random state with each qudit entry drawn from the uniform distribution in the interval $[0, \tilde{f})$. The state is then normalized, and $E_F(\psi) + E_W(\psi)$ is minimized directly by N_{steps} gradient descent steps. We also introduce a further hyperparameter, Patience, which stops the algorithm early if no improvement in E_{Potts} [Eq. (1)] is seen after a certain number gradient descent steps.

In both cases, we sample over N_{runs} runs of the algorithm and each run is stopped if $E_{\text{Potts}} = 0$ is found. We consider the best-found solutions and their frequency as indicators of the performance of the two algorithms.

To perform the gradient descent, we use the Adam optimizer [41] with learning η , specifically the PyTorch implementation [42]. All other parameters in the Adam optimizer were set to their PyTorch default values. Also, note that we provide pseudocode for the two algorithms in Appendix A, an overview of runtimes for the calculations presented later in Appendix B, and a detailed list and description of all hyperparameters specific to the algorithms in Appendix C.

III. BENCHMARK

We benchmark our algorithms by considering several standard GCP instances used to test other methods [32–34, 43] and comparing to results in the literature [32–34]. First, we consider Myciel and Queen graphs from the COLOR data set [44] and the citation networks Cora and PubMed [45–47]. The Myciel and Queen graphs, based on Mycielski transformation and $n \times m$ chessboards respectively, have up to 169 nodes and densities between 16.91% and 53.33%. In contrast, the Cora and PubMed graphs have 2708 and 19717 nodes and densities of 0.15% and 0.02%. Note that a more thorough description of the datasets can be found in Ref. [33].

We then consider three graphs from the SNAP library [48]: The Facebook dataset based on social circles from Facebook, the Wikipedia vote dataset based on Wikipedia voting data, and the email-Eu-core data set based on email communication between members of a research institute. For these three graphs, it is known [32] that relatively large numbers of colors are required to solve the problems (we use 70, 22, and 19) and thus they provide interesting challenges for our algorithms. In all cases, we consider undirected graphs, and we remove duplicated edges and any nonconnected nodes when preprocessing the data.

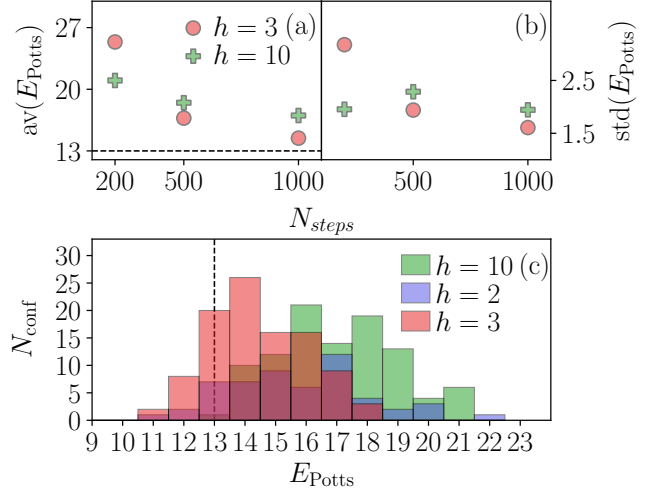


FIG. 3. (a) Average of color conflicts corresponding to the Potts energy from Eq. (1) for the queen11-11 graph as a function of total number of time steps N_{steps} . We use $c = 11$ colors, $f = 0.0$, and different h . For the calculations we use $N_{\text{runs}} = 100$. The black dashed line is at 13, the best solution presented in Ref. [34]. (b) Standard deviation for the data shown in (a). (c) Histogram showing the number, N_{conf} , of times a certain energy was found for $N_{\text{steps}} = 1000$. Also here, the dashed line is at 13. All parameters not specified are provided in Appendix C.

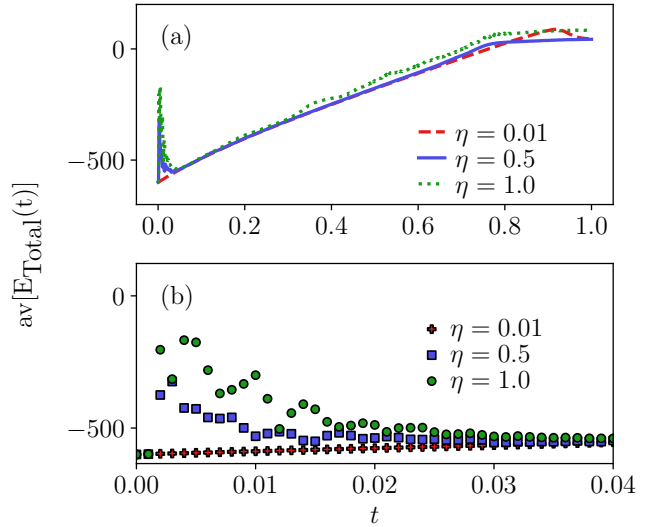


FIG. 4. (a) Total energy for different values of the learning rate η for $N_{\text{steps}} = 1000$ and $c = 11$. (b) Same data as in (a) but on a shorter time scale and showing the individual data points. All other parameters can be found in Appendix C.

A. Influence of the hyperparameters

First, we illustrate how some of the hyperparameters in the QdLQA algorithm influence the final solutions. As an example, we show results for the queen11-11 graph.

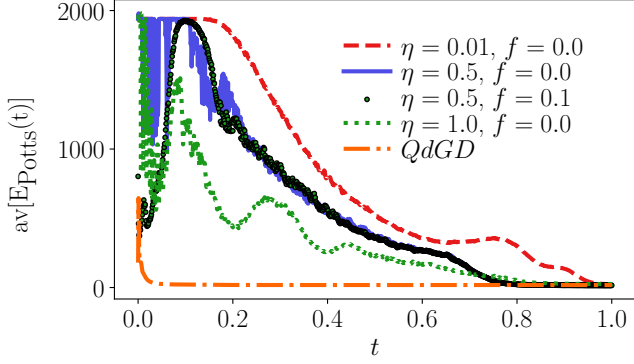


FIG. 5. Average number of color clashes $[E_{\text{Potts}}$ from Eq. (1)] for the same parameters as in Fig. 4. The orange line additionally shows the data calculated using QdGD and the minimum values of E_{Potts} reached in the simulations were 19 ($\eta = 0.01$, $f = 0.0$), 11 ($\eta = 0.5$, $f = 0.0$), 10 ($\eta = 0.5$, $f = 0.1$), 12 ($\eta = 0.01$, $f = 0.0$), and 13 (QdGD) using $N_{\text{runs}} = 100$. All other parameters can be found in Appendix C.

In Fig. 3(a), we demonstrate how the average value of the Potts energy depends on the number of time steps, N_{steps} , for different values of h [which generates randomness in $E_F(\psi)$ in Eq. (9)]. Increasing the number of steps improves the average quality of the solutions. The black dashed line shows the best value for E_{Potts} reported in Ref [34]. Similar behavior is seen for the standard deviation of the $h = 3$ data in Fig. 3(b). In addition, we see that for large N_{steps} , the $h = 10$ data have a larger standard deviation and average value than $h = 3$. We observe that having a finite h helps the algorithm escape from local minima, but if h is chosen too large, this comes at the cost of more low-quality solutions. This is further illustrated in Fig. 3(c), where we show a histogram of the number of times, N_{conf} , a configuration with a certain energy is found. Again, the black dashed line shows the best value for E_{Potts} reported in Ref [34]. There, we see how also choosing $h = 2$ increases the fluctuations and leads to the algorithm getting stuck more frequently in a bad local minimum. In 42 instances, the solution found by $h = 2$ was outside the scale of the figure. Thus, the $h = 2$ data are also not plotted in Fig. 3(a).

We now show an example of how the cost function E_{Total} and the Potts energy E_{Potts} behave during the quantum adiabatic evolution in QdLQA. In Fig. 4(a), we plot E_{Total} for the queen11-11 graph as a function of t averaged over $N_{\text{runs}} = 100$ runs for different learning rates η . There, we see that a large learning rate leads to an initial jump in the cost function, which increases when η is increased. This can be seen better in Fig. 4(b), where the individual data points of the initial dynamics are plotted. We attribute this jump to the fact that we start in the global minimum at $t = 0$, which remains close to the minimum for small increments in t . Thus, doing a gradient descent step with a large learning rate will make us "overshoot" and take us away from the presumptive

minima. However, we see that the $\eta = 0.5$ curve quickly relaxes to the $\eta = 0.01$ curve for a large part of the time evolution. In contrast, the $\eta = 1.0$ curve also shows large fluctuations for later times.

Figure 5 shows E_{Potts} for the same parameters as Fig. 4. Here, one can see large fluctuations in the energies for $\eta = 0.5$ and $\eta = 1.0$, but which decrease when t is increased. The initial decay coincides with the jump in Fig. 4 and can be understood as follows: In the initial state, all qudits are initialized equally, and thus, E_{Potts} has maximum value. If one now goes away from the minimum, this leads to different qudits taking on different colors, and thus the value of E_{Potts} will decrease. If one perturbs the initial state (the $f = 0.1$ setting is illustrated by the black dots), the colors at different qudits will differ, and thus, E_{Potts} is smaller at $t = 0$. However, at short times, the cost function will still favor all colors being the same [the ground state of Eq. (8)], and thus E_{Potts} increases. For larger times, it then closely resembles the curve for the unperturbed initial state (blue solid line) as it decreases. The orange curve in Fig. 5 shows E_{Potts} for the QdGD algorithm (as a function of gradient descent steps $n = N_{\text{steps}}t$). Notably, this decays rapidly, which allows for a good solution to be found much earlier. When stopping the QdGD after having 100 gradient descent steps without improving E_{Potts} , the average number of steps was $n = 259.28$ (corresponding to $t \approx 0.26$).

Lastly, we want to point out another interesting feature of our approach. In most applications (e.g., often when training neural networks), optimizers like Adam update their internal parameters based on previous gradient descent steps for a fixed cost function. Here, in contrast, the parameters are updated based on the energy landscape at previous times (due to the varying cost function).

B. Comparing with other methods

We now compare the solutions found by our algorithms to previously published results using graph neural networks in Refs. [33, 34]. In Table I, we show the energies of the best configurations found with QdLQA and QdGD ($E_{\text{Potts}}^{\text{QdLQA}}$ and $E_{\text{Potts}}^{\text{QdGD}}$, respectively) and the fraction of the runs that found that energy ($p_{\text{min}}^{\text{QdLQA}}$ and $p_{\text{min}}^{\text{QdGD}}$) for different graphs. Furthermore, we show the corresponding values reported in Refs. [33, 34] obtained using a Tabucol [49] and a greedy algorithm, a physics-inspired graph neural network solver (PI-SAGE) [33], and graph neural networks using a first-order negative message passing strategy (GNN-1N) [34]. Note that if $E_{\text{Potts}} = 0$, the graph was colored with no conflicts, otherwise, $E_{\text{Potts}} > 0$ gives the remaining number of color clashes. For all the graphs except queen11-11, queen13-13, Cora, and PubMed, both our algorithms find the optimal solution. However, some graphs seem to be particularly hard. For example, for the queen8-8 and queen9-9

Graph	Nodes	Edges	Colors (c)	Tabucol	PI-CGN	PI-SAGE	GNN-1N	$E_{\text{Potts}}^{\text{QdLQA}}$	$E_{\text{Potts}}^{\text{QdGD}}$	$p_{\min}^{\text{QdLQA}}$	$p_{\min}^{\text{QdGD}}$
myciel5	47	236	6	0	0	0	0	0	0	100/100	100/100
myciel6	95	755	7	0	0	0	0	0	0	38/100	97/100
queen5-5	25	160	5	0	0	0	0	0	0	100/100	68/100
queen6-6	36	290	7	0	1	0	0	0	0	12/100	7/100
queen7-7	49	476	7	0	8	0	0	0	0	17/100	8/100
queen8-8	64	728	9	0	6	1	1	0	0	6/100	2/100
queen9-9	81	1056	10	0	13	1	1	0	0	3/100	1/100
queen8-12	98	1368	12	0	10	0	0	0	0	27/100	41/100
queen11-11	121	1980	11	20	37	13*	13	10	13	1/100	2/100
queen13-13	169	3328	13	25	61	26	15	12	15	2/100	1/100
Cora	2708	5429	5	0	1	0	X	1	0	3/100	1/100
Pubmed	19717	44338	8	NA	13	17	X	15	27	3/100	3/100

TABLE I. Datasets from [44] and the citation graphs [45–47] introduced in the main text. Here, “Colors” represents the number of colors used in the calculations, while the following values are related to the cost function obtained after optimization by various methods, such that zero is optimum. Values given for Tabucol [33], PI-CGN [33], PI-SAGE [33], and GNN-1N [34] represent other algorithms, whilst the quantities related to $E_{\text{Potts}}^{\text{QdLQA}}$ and $E_{\text{Potts}}^{\text{QdGD}}$ show the best solution found by the corresponding method in N_{runs} runs. The parameters $p_{\min}^{\text{QdLQA}}$ and $p_{\min}^{\text{QdGD}}$ give the fractions, $N_{\min}/N_{\text{runs}}$, indicating how often, $N_{\min}$, the solution was found in the N_{runs} runs. The best solution found by any of the algorithms is indicated in bold face, NA means that no solution was found within 24 h [33], and X means no value was provided in Ref. [34]. For Cora and PubMed, the choice of c follows Refs. [33, 43]. * 17 was reported in [33] but we find 13 in our own calculations, see Appendix B.

graphs, both algorithms only find the optimal solutions in less than 10 of the 100 runs. For queen11-11 and queen13-13, QdLQA finds better solutions than those reported in Refs. [33, 34]. In the case of queen11-11, it additionally finds $E_{\text{Potts}} = 11$ in three of the 100 runs, and both $E_{\text{Potts}} = 12$ and $E_{\text{Potts}} = 13$ in nine of the 100 runs each. For queen13-13, it additionally finds $E_{\text{Potts}} = 13$ in two, $E_{\text{Potts}} = 14$ in two, and $E_{\text{Potts}} = 15$ in seven of the 100 runs.

By systematically increasing the number of colors used, we can get upper bounds for the chromatic number $\chi(G)$ which we compare to those reported in Ref. [33]. For queen11-11, both our algorithms find $\chi_{\text{Upper}}(G) = 13$ with $p_{\min}^{\text{QdLQA}} = 37/100$ and $p_{\min}^{\text{QdGD}} = 22/100$ (14 is found by the PI-SAGE algorithm in Ref. [33], 11 is optimal) and for queen13-13, both find $\chi_{\text{Upper}}(G) = 15$ with $p_{\min}^{\text{QdLQA}} = 43/100$ and $p_{\min}^{\text{QdGD}} = 3/100$ (17 is found by the PI-SAGE algorithm in Ref. [33], 13 is optimal).

For the Cora graph, only the QdGD algorithm finds the $E_{\text{Potts}} = 0$ solution, and only in one of the 100 runs. QdLQA finds $E_{\text{Potts}} = 1$ configurations in three and $E_{\text{Potts}} = 2$ in 20 of the 100 runs, but requires a comparatively large h to do so (we use $h = 10$). In this case, one must compromise between finding $E_{\text{Potts}} = 1$ a few times at the cost of obtaining many bad solutions. For example, using $h = 3$ instead, we get $E_{\text{Potts}} = 2$ in 58 of 100 runs, but never $E_{\text{Potts}} = 1$.

For the PubMed graph, neither algorithm finds better solutions than the best one reported in Ref. [33], but QdLQA seems to do significantly better than QdGD. In

addition, in contrast to the other datasets in Table I, we do not fix the node with the largest number of edges for the PubMed graph. Rather, we fix a node with one edge as this improves the results. For $N_{\text{runs}} = 100$, QdLQA improves from finding $E_{\text{Potts}}^{\text{QdLQA}} = 17$ three times to finding $E_{\text{Potts}}^{\text{QdLQA}} = 15$, 16, and 17 three times, once, and seven times, respectively. QdGD improves from finding $E_{\text{Potts}}^{\text{QdGD}} = 28$ twice to finding $E_{\text{Potts}}^{\text{QdGD}} = 27$ three times. This indicates that cleverly fixing one (or more) nodes might lead to further improvements.

Note that we provide a comprehensive overview of the average runtimes, computer architectures used, and a more thorough comparison to PI-SAGE in Appendix B and a list of parameters used in Appendix C.

C. Benchmarking using large number of colors

Finally, we test our algorithms on graphs where it is known that relatively large numbers of colors are needed to solve the problems [32]. In Table II, we show results for the previously mentioned email-Eu-core, Facebook, and Wiki-vote datasets together with the corresponding upper and lower bounds provided in Ref. [32] (the problem was solved for email-Eu-core and Facebook). In our simulations, we use the number of colors equal to the upper bounds provided in Ref. [32] (19 for email-Eu-core, 70 for Facebook, and 22 for Wiki-vote), and again, $E_{\text{Potts}}^{\text{QdLQA}}$

Graph	Nodes	Edges	$\chi_{Lower}(G)$	$\chi_{Upper}(G)$	E_{Potts}^{QdLQA}	E_{Potts}^{QdGD}	p_{min}^{QdLQA}	p_{min}^{QdGD}
email-Eu-core	986	16064	19	19	26	30	1/100	1/100
Facebook	4039	88234	70	70	25	22	1/100	1/100
Wiki-vote	7115	103689	19	22	192	207	1/100	1/100

TABLE II. Minimum energy, $\min(E_{Potts})$, found for the email-Eu-core, Facebook, and Wiki-vote datasets [48] together with the upper and lower bounds for the chromatic number $\chi(G)$ reported in Ref. [32]. In our simulations, we set the number of colors $c = \chi_{Upper}(G)$ from [32]. The parameters used can be found in Appendix C.

and E_{Potts}^{QdGS} give the number of conflicts in the best solution found. Even though neither algorithm solves the problem completely, they are clearly capable of providing relatively good solutions (quantified by the normalized error defined as the energy divided by number of edges, $\epsilon = E_{Potts}/|\mathcal{E}|$ [33]). For the email-Eu-core, Facebook, and Wiki-vote datasets, $\epsilon \approx 0.0016$ (0.0019), 0.00028 (0.00025) and 0.0019 (0.002) for QdLQA (QdGS). This demonstrates the ability of the algorithms to find good solutions for relatively large graphs requiring many colors on a simple laptop or desktop (see Appendix B for details).

IV. CONCLUSION

In this work, we have proposed two quantum-inspired algorithms to efficiently solve integer optimization problems. In our approach, we map nodes to qudits parameterized by c -dimensional spherical coordinates and perform either local quantum annealing (QdLQA) or a gradient descent (QdGD).

We compared our results for a range of GCPs to some of the state-of-the-art methods in the literature [33, 34], and both QdLQA and QdGD find equally good or better solutions for all of the test cases, with the exception of the PubMed graph. We also tested the algorithms on graphs with large chromatic numbers [32]. Even though neither algorithm solved these problems completely, they still provided reasonable-quality solutions (with normalized error $\epsilon \approx 10^{-3}$) using modest computing resources such as a desktop or a laptop. Notably, QdLQA almost always provided equally good or better solutions than QdGD (with the exception of the Cora and the Facebook graph). However, QdGD was heuristically found to be more efficient, since it minimizes the desired cost function directly. We refer the reader to Appendix B for further details.

There are many compelling possible extensions to our work. For example, each local Hilbert space dimension can be set manually, and thus, the approach can easily be extended to integer optimization problems requiring different numbers of states on each node. In general, it would also be interesting to do a detailed comparison between local quantum annealing and simulated (thermal) annealing; however, it seems that for the GCP, the latter suffers convergence and scalability issues [50–52].

Furthermore, the interpretation of a vector on a sphere might provide new insights into why certain solutions are preferred (which can also be done with an analysis of the solutions and the interactions due to h). In addition, there is plenty of room for general optimizations, hyperparameter tuning, and different choices (e.g., initial configurations). We have chosen to initialize the nodes in the ground states of $-\hat{L}_i^x$ since this connects with quantum annealing for qudit systems. However, other initial states, e.g., all colors equally populated, might improve the results further. It will also be important to investigate if the algorithms can provide competitive solutions to GCPs containing millions of nodes and see if they can be generalized to other problems, such as large-scale graph-based machine learning tasks [53, 54].

Finally, we note that by restricting the cost function to be a quantum Hamiltonian (see, for example, Ref. [7]), our method is similar to simulating quantum annealing in a qudit system (without entanglement). Therefore, one could also investigate if allowing for entanglement in the system might help the algorithms find better solutions. One approach would be to use tensor networks with a bond dimension larger than 1 (see, for example, Ref. [23]). However, this comes with significant computational overhead and might be unfeasible to scale to industrial-size problems.

Data availability: The data supporting our results can be found at <https://zenodo.org/records/11483787>.

ACKNOWLEDGMENT

We acknowledge useful discussions with Márcio M. Taddei and Benjamin Schiffer and the support from the Government of Spain (European Union NextGenerationEU PRTR-C17.I1, Quantum in Spain, Severo Ochoa CEX2019-000910-S and FUNQIP), Fundació Cellex, Fundació Mir-Puig, the EU (NEQST 101080086, Quanterra Veriqtas, PASQuanS2.1 101113690), the ERC AdG CERQUTE, the AXA Chair in Quantum Information Science and Generalitat de Catalunya (CERCA program). This project has received funding from the European Union’s Horizon 2020 research and innovation programme under the Marie Skłodowska-Curie grant Agreement No 847517.

Algorithm 1: QdLQA

For the specifics regarding the initialization of the state ψ and formulation of the cost function $E_{\text{Total}}(\psi, t)$, see the main text.

```

Load  $\alpha, f, N_{\text{runs}}, N_{\text{steps}}$ 
 $e_{\text{list}}$  is an empty list
for  $i = 1$  to  $N_{\text{runs}}$  do
    Initialize the state  $\psi$  that depend on parameters  $\vec{\phi}$ 
    with perturbation  $f$ 
     $t \leftarrow 0$ 
     $e_{\text{best}} \leftarrow 1000000$ 
    while  $t < 1$  do
        Minimize  $E_{\text{Total}}(\psi, t)$  with  $\alpha$  steps
         $e_{\text{Potts}} \leftarrow E_{\text{Potts}}(\psi)$ 
         $e_{\text{best}} \leftarrow \min(e_{\text{best}}, e_{\text{Potts}})$ 
        if  $e_{\text{best}} = 0$  then
            break
        end
         $t \leftarrow t + 1/N_{\text{steps}}$ 
    end
    Add  $e_{\text{best}}$  to  $e_{\text{list}}$ 
end
Return  $e_{\text{list}}$ 

```

Algorithm 2: QdGD

For the specifics regarding the initialization of the state ψ and formulation of the cost function $E_{\text{Total}}(\psi)$, see the main text.

```

Initialize  $\tilde{f}, N_{\text{runs}}, N_{\text{steps}}, \text{Patience}$ 
 $e_{\text{list}}$  is an empty list
for  $i = 1$  to  $N_{\text{runs}}$  do
    Initialize the state  $\psi$  that depend on parameters  $\vec{\phi}$ 
    with perturbation  $\tilde{f}$ 
     $e_{\text{best}} \leftarrow 1000000$ 
    for  $j = 1$  to  $N_{\text{steps}}$  do
        Minimize  $E_{\text{Total}}(\psi)$  with one step
         $e_{\text{Potts}} \leftarrow E_{\text{Potts}}(\psi)$ 
         $e_{\text{best}} \leftarrow \min(e_{\text{best}}, e_{\text{Potts}})$ 
        if  $e_{\text{best}} = 0$  or no improvement then
            break
        end
    end
    Add  $e_{\text{best}}$  to  $e_{\text{list}}$ 
end
Return  $e_{\text{list}}$ 

```

Appendix A: Pseudocode

Here, we present pseudocode for the algorithms QdLQA and QdGD (Algorithm 1 and Algorithm 2, respectively). In both algorithms, we first load the graph's nodes and edges and initialize the hyperparameters and an empty list e_{list} . For each of the N_{runs} of the algorithms, we initialize the state ψ which depends on the parameters $\vec{\phi}$. In practice, this can, for example, be a list of length $|V|$ (the number of nodes), where each entry is a vector and depends on elements of $\vec{\phi}$ as written in Eq. (6). In this work, the vectors are initialized to the ground state of $-\hat{L}_i^x$ for QdLQA and random values for QdGD. However, other initialization schemes are possible. For QdLQA, we then let the time t go from 0 to 1, and at each time step, we minimize a cost function $E_{\text{Total}}(\psi, t)$ using α gradient descent steps. We then compute the classical energy e_{Potts} (we use upper case letters for functions and lower-case letters for variables). At the end of each run, the lowest energy is added to e_{list} , and finally, e_{list} is returned. This can then be used to extract the values in Table I and Table II. For QdGD, we proceed in a similar manner. Here, however, the cost function does not depend on time, and we also stop the algorithm if no improvement in classical energy is found based on the hyperparameter Patience.

The pseudocode for the cost function we use for QdLQA is shown in Algorithm 3 (for QdGD it is just the same function evaluated at $t = 1$). $\hat{L}^x$ is a matrix of size $c \times c$ and is given by the x angular momentum operator (we drop the subindex i since it is site independent in our implementation), see Sec. IIB, and j_{max} is a fixed qudit initialized to one color. Note that the j_{max} qudit can also

Algorithm 3: $E_{\text{Total}}(t, \psi)$

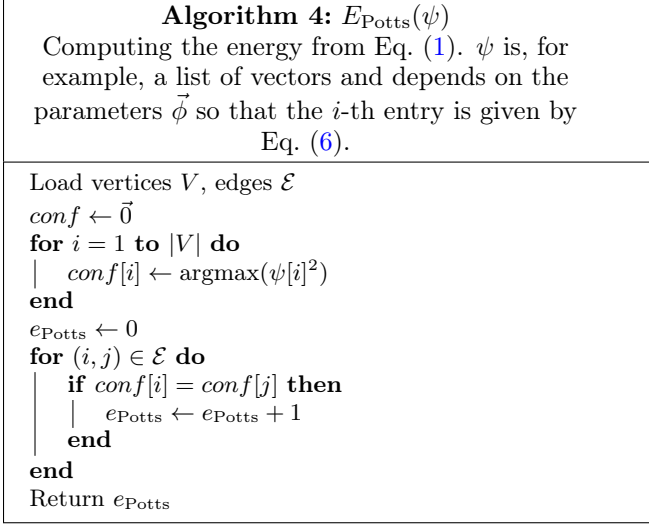
Computing the cost function from Eq. (11). t is a floating point and ψ is, for example, a list of vectors and depends on the parameters $\vec{\phi}$ so that the i -th entry is given by Eq. (6).

```

Load vertices  $V$ , edges  $\mathcal{E}$ , matrix  $\hat{L}^x, j_{\text{max}}, \gamma, h$ 
 $e_{\text{F}} \leftarrow 0$ 
 $e_{\text{W}} \leftarrow 0$ 
 $e_{\text{I}} \leftarrow 0$ 
for  $(i, j) \in \mathcal{E}$  do
     $h_{i,j} \leftarrow$  sampled from uniform distribution  $[0, h)$ 
     $e_{\text{F}} \leftarrow e_{\text{F}} + \psi[i]^2 \cdot \psi[j]^2 (\mathcal{E}[i, j] + h_{i,j})$ 
end
for  $l = 1$  to  $|V|$  do
     $e_{\text{W}} \leftarrow e_{\text{W}} + \gamma \psi[l]^2 \cdot \log(\psi[l]^2)$ 
end
for  $n = 1$  to  $|V|$  do
    if  $n \neq j_{\text{max}}$  then
         $e_{\text{I}} \leftarrow e_{\text{I}} - \psi[n]^T \cdot L^x \cdot \psi[n]$ 
    end
end
Return  $(1 - t)e_{\text{I}} + t(e_{\text{F}} + e_{\text{W}})$ 

```

be emitted in the second for loop as it only contributes with a constant term. In the algorithm, squaring and taking the logarithm are done componentwise. Lastly, we show how we extract the final color configuration in Algorithm 4. There, conf is the zero vector of size $|V|$ and argmax gives the index of the component with the largest magnitude of the vector $\psi[i]^2$.



Appendix B: Computer architectures and runtimes

All computations, except those on the PubMed, Facebook, and Wiki-vote graphs, were run on a laptop with an 11th-generation Intel(R) Core(TM) i5-1135G7 @ 2.40GHz processor, whereas the latter ran on a desktop with an 11th-generation Intel(R) Core(TM) i5-11500 @ 2.70GHz processor. In Fig. 6, we plot the average computation time for one run for the different graphs for the parameters used in this work; see Appendix C. Note that the parameter α varies (for queen13-13), so the figure is an illustration of the cost of obtaining the data presented, rather than fairly comparing the computational cost for the different graphs. Figure 6(a) shows the average time for one run of the algorithms for the different graphs on the laptop and Figure 6(b) on a stationary computer [55]. As expected, the run time is significantly impacted by the Patience hyperparameter for QdGD, which allows the algorithm to stop early.

For a more thorough comparison between the graph neural network approaches, QdLQA, and QdGD, we run the PI-SAGE algorithm from Ref. [33] on the previously mentioned laptop for several datasets. For the calculations, we use the code available at [56] and the parameters reported in Ref. [33]. In Fig. 7, we show the average runtimes; as the problem sizes increase, the QdLQA and QdGD run significantly faster (note that this might change on different computer architectures, where, for example, GPUs are available). In addition, the QdLQA and QdGD only optimize $(|V| - 1)(c - 1)$ parameters (the minus one in $(|V| - 1)$ comes from already assigning one color to one qudit in the initial state), which is a fraction of what, for example, PI-SAGE requires (even though the number of parameters in principle scales as $\sim c + |V|$). For example, for the queen5-5, queen9-9, and queen11-11 datasets, our methods have 96, 720, and 1200 parameters, whereas the PI-SAGE method uses 7210, 12663, and 13411 parameters (using the hyperparameters from

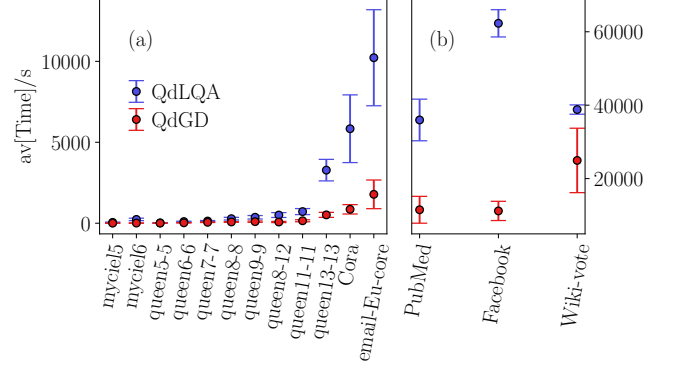


FIG. 6. Average computation times on (a) a laptop and (b) a stationary computer for N_{runs} of QdLQA and QdGD for different graphs. The error bars correspond to the standard deviation of the data.

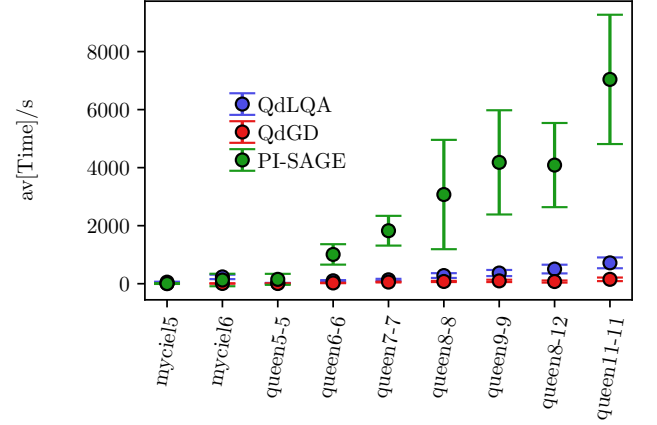


FIG. 7. Average computation times on the laptop described in the main text for QdLQA, QdGD, and PI-SAGE. For all algorithms, we use $N_{\text{runs}} = 100$.

Ref. [33]).

Lastly, we emphasize that the graph neural network approaches also might need $N_{\text{runs}} > 1$ to find an optimal solution. By introducing $p_{\min}^{\text{PI-SAGE}} = N_{\min}/N_{\text{runs}}$, we get the following tuples (dataset, $E_{\min}^{\text{PI-SAGE}}$, $p_{\min}^{\text{PI-SAGE}}$): (myciel5, 0, 100/100), (myciel6, 0, 100/100), (queen5-5, 0, 85/100), (queen6-6, 0, 17/100), (queen7-7, 0, 1/100), (queen8-8, 1, 3/100), (queen9-9, 1, 6/100), (queen8-12, 2, 6/100), and (queen11-11, 13, 1/100) in our calculations (the last result actually improves on what was reported in Ref. [33]).

Appendix C: Hyperparameters

There are several hyperparameters needed in both the

Hyperparameter	Algorithm	Description:
N_{steps}	QdLQA	Number of steps used when going from $t = 0$ to $t = 1$ in the quantum adiabatic evolution
N_{steps}	QdGD	Number of gradient descent steps
γ	Both	Prefactor in $E_W(\psi)$ from Eq. (10)
α	QdLQA	Determines the number of gradient descent steps at each time step t
η	Both	Learning rate in the Adam optimizer
f	QdLQA	Determines the perturbation in the angles in the initial state
h	Both	Determines the perturbation in the interactions in the Potts model, see Eq. (9)
N_{runs}	Both	Number of times we run the algorithm
Patience	QdGD	If no improvement in E_{Potts} is seen after this many gradient descent steps, the algorithm stops
$\tilde{f}$	QdGD	Determines the sampling interval for each qudit in the initial state

TABLE III. Summary of the hyperparameters for the QdLQA and QdGD algorithms.

	QdLQA								QdGD						
Graph	N_{steps}	γ	α	η	f	h	N_{runs}		N_{steps}	γ	Patience	η	$\tilde{f}$	h	N_{runs}
myciel5	1000	1.0	1	0.5	0.0	3.0	100		1000	1.0	100	0.5	1.0	3.0	100
myciel6	1000	1.0	1	0.5	0.0	3.0	100		1000	1.0	100	0.5	1.0	3.0	100
queen5-5	1000	1.0	1	0.5	0.0	3.0	100		1000	1.0	100	0.5	1.0	3.0	100
queen6-6	1000	1.0	1	0.5	0.0	3.0	100		1000	1.0	100	0.5	1.0	3.0	100
queen7-7	1000	1.0	1	0.5	0.0	3.0	100		1000	1.0	100	1.0	1.0	1.0	100
queen8-8	1000	1.0	1	0.5	0.1	3.0	100		1000	0.5	100	1.0	1.0	1.0	100
queen9-9	1000	1.0	1	0.5	0.0	3.0	100		1000	0.5	100	0.1	1.0	1.0	100
queen8-12	1000	1.0	1	0.5	0.0	3.0	100		1000	1.0	100	0.5	1.0	3.0	100
queen11-11	1000	1.0	1	0.5	0.1	3.0	100		1000	1.0	100	0.5	1.0	3.0	100
queen13-13	1000	0.75	*	0.5	0.0	3.0	100		3175	1.0	100	0.1	1.0	3.0	100
Cora	1000	0.25	1	0.5	0.1	10.0	100		1000	0.5	100	1.0	1.0	3.0	100
Pubmed	1000	0.1	1	0.1	0.5	3.0	100		1000	0.5	100	0.5	1.0	3.0	100
email-Eu-core	1500	0.5	1	0.1	0.1	3.0	100		1000	0.5	100	0.1	1.0	1.0	100
Facebook	1000	0.1	1	0.1	0.1	3.0	100		1000	0.1	100	0.1	1.0	3.0	100
Wiki-vote	1000	0.1	1	0.5	0.1	3.0	100		1000	0.1	100	0.5	1.0	3.0	100

TABLE IV. Hyperparameters used to compute the data in Tables I and II. * For queen13-13, we conduct $\text{round}(e^{2t})$ gradient descent steps ($\text{round}()$ indicates that we round to the nearest integer) at each point in time (maximum 7).

QdLQA and the QdGD algorithms. These and their effects are summarized in Table III. In addition, the Adam optimizer has further hyperparameters that could be tuned (here, we only adjust the learning rate), however, in this work, they are left to their default values in PyTorch.

In Table IV, we show the hyperparameters used to produce the data in Tables I and II. Initially, we choose the

default values $\eta = 0.5$, $h = 3.0$, $\alpha = 1$, $N_{\text{steps}} = 1000$ and $\gamma = 1.0$ which work well for many of the graphs. When the algorithms neither solved the problem nor achieved better solutions than the graph neural network approaches, we either tuned the parameters manually or did a grid search. Note that for queen13-13, we choose $N_{\text{steps}} = 3175$ for QdGD so that we get equally as many gradient descent steps as in QdLQA.

[1] J. Kallrath, Solving planning and design problems in the process industry using mixed integer and global optimization, *Ann. Oper. Res.* **140**, 339 (2005).

[2] W.-Y. Ku and J. C. Beck, Mixed integer programming

- models for job shop scheduling: A computational analysis, *Comput. Oper. Res.* **73**, 165 (2016).
- [3] F. He and R. Qu, A two-stage stochastic mixed-integer program modelling and hybrid solution approach to portfolio selection problems, *Inf. Sci.* **289**, 190 (2014).
 - [4] R. M. R. Lewis, *A Guide to Graph Colouring: Algorithms and Applications* (Springer, Heidelberg, 2016).
 - [5] M. Garey, D. Johnson, and L. Stockmeyer, Some simplified NP-complete graph problems, *Theor. Comput. Sci.* **1**, 237 (1976).
 - [6] N. Barnier and P. Brisset, Graph coloring for air traffic flow management, *Ann. Oper. Res.* **130**, 163 (2004).
 - [7] Y. Deller, S. Schmitt, M. Lewenstein, S. Lenk, M. Federer, F. Jendrzewski, P. Hauke, and V. Kasper, Quantum approximate optimization algorithm for qudit systems, *Phys. Rev. A* **107**, 062410 (2023).
 - [8] E. Farhi, J. Goldstone, and S. Gutmann, A quantum approximate optimization algorithm, [arXiv:1411.4028](#) (2014).
 - [9] S. Hadfield, Z. Wang, B. O’Gorman, E. G. Rieffel, D. Venturelli, and R. Biswas, From the quantum approximate optimization algorithm to a quantum alternating operator ansatz, *Algorithms* **12** (2019).
 - [10] Y.-H. Oh, H. Mohammadbagherpoor, P. Dreher, A. Singh, X. Yu, and A. J. Rindos, Solving multi-coloring combinatorial optimization problems using hybrid quantum algorithms, [arXiv:1911.00595](#) (2019).
 - [11] P. Hauke, H. G. Katzgraber, W. Lechner, H. Nishimori, and W. D. Oliver, Perspectives of quantum annealing: methods and implementations, *Rep. Prog. Phys.* **83**, 054401 (2020).
 - [12] B. Pokharel, Z. Gonzalez Izquierdo, P. A. Lott, E. Strbac, K. Osiewalski, E. Papathanasiou, A. Kondratyev, D. Venturelli, and E. Rieffel, Inter-generational comparison of quantum annealers in solving hard scheduling problems, *Quantum Inf. Process.* **22**, 364 (2023).
 - [13] S. Yarkoni, E. Raponi, T. Bäck, and S. Schmitt, Quantum annealing for industry applications: introduction and review, *Rep. Prog. Phys.* **85**, 104001 (2022).
 - [14] A. Lucas, Ising formulations of many NP problems, *Front. Phys.* **2** (2014).
 - [15] F. Glover, G. Kochenberger, R. Hennig, and Y. Du, Quantum bridge analytics I: a tutorial on formulating and using QUBO models, *Ann. Oper. Res.* **314**, 141 (2022).
 - [16] H.-H. Lu, Z. Hu, M. S. Alshaykh, A. J. Moore, Y. Wang, P. Imany, A. M. Weiner, and S. Kais, Quantum phase estimation with time-frequency qudits in a single photon, *Adv. Quantum Technol.* **3**, 1900074 (2020).
 - [17] Y. Wang, Z. Hu, B. C. Sanders, and S. Kais, Qudits and high-dimensional quantum computing, *Front. Phys.* **8** (2020).
 - [18] M. Ringbauer, M. Meth, L. Postler, R. Stricker, R. Blatt, P. Schindler, and T. Monz, A universal qudit quantum processor with trapped ions, *Nat. Phys.* **18**, 1053 (2022).
 - [19] G. Bottrill, M. Pandey, and O. Di Matteo, Exploring the potential of qutrits for quantum optimization of graph coloring, *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*, volume 01, 177–183 (2023).
 - [20] J. A. Smolin and G. Smith, Classical signature of quantum annealing, *Front. Phys.* **2** (2014).
 - [21] M. T. Veszeli and G. Vattay, Mean field approximation for solving QUBO problems, *PLOS ONE* **17**, 1 (2022).
 - [22] J. Bowles, A. Dauphin, P. Huembeli, J. Martinez, and A. Acín, Quadratic unconstrained binary optimization via quantum-inspired annealing, *Phys. Rev. Appl.* **18**, 034016 (2022).
 - [23] B. Bauer, L. Wang, I. Pizorn, and M. Troyer, Entanglement as a resource in adiabatic quantum optimization, [arXiv:1501.06914](#) (2015).
 - [24] S. Mugel, C. Kuchkovsky, E. Sánchez, S. Fernández-Lorenzo, J. Luis-Hita, E. Lizaso, and R. Orús, Dynamic portfolio optimization with real datasets using quantum processors and quantum-inspired tensor networks, *Phys. Rev. Res.* **4**, 013006 (2022).
 - [25] J. Gomes, K. A. McKiernan, P. Eastman, and V. S. Pande, Classical quantum optimization with neural network quantum states, [arXiv:1910.10675v1](#) (2019).
 - [26] T. Zhao, G. Carleo, J. Stokes, and S. Veerapaneni, Natural evolution strategies and variational Monte Carlo, *Mach. Learn.: Sci. Technol.* **2**, 02LT01 (2020).
 - [27] M. J. A. Schuetz, J. K. Brubaker, and H. G. Katzgraber, Combinatorial optimization with physics-inspired graph neural networks, *Nat. Mach. Intell.* **4**, 367–377 (2022).
 - [28] S. A. Khandoker, J. M. Abedin, and M. Hibat-Allah, Supplementing recurrent neural networks with annealing to solve combinatorial optimization problems, *Mach. Learn.: Sci. Technol.* **4**, 015026 (2023).
 - [29] E. S. Tiunov, A. E. Ulanov, and A. I. Lvovsky, Annealing by simulating the coherent Ising machine, *Opt. Express* **27**, 10288 (2019).
 - [30] H. Goto, K. Tatsumura, and A. R. Dixon, Combinatorial optimization by simulating adiabatic bifurcations in nonlinear Hamiltonian systems, *Sci. Adv.* **5**, eaav2372 (2019).
 - [31] H. Goto, K. Endo, M. Suzuki, Y. Sakai, T. Kanao, Y. Hamakawa, R. Hidaka, M. Yamasaki, and K. Tatsumura, High-performance combinatorial optimization based on classical mechanics, *Sci. Adv.* **7**, eabe7953 (2021).
 - [32] E. Hebrard and G. Katsirelos, A hybrid approach for exact coloring of massive graphs, *Integration of Constraint Programming, Artificial Intelligence, and Operations Research*, edited by L.-M. Rousseau and K. Stergiou, 374–390 (Springer International Publishing, Cham, 2019).
 - [33] M. J. A. Schuetz, J. K. Brubaker, Z. Zhu, and H. G. Katzgraber, Graph coloring with physics-inspired graph neural networks, *Phys. Rev. Res.* **4**, 043131 (2022).
 - [34] X. Wang, X. Yan, and Y. Jin, A graph neural network with negative message passing and uniformity maximization for graph coloring, *Complex Intell. Syst.* **10**, 4445 (2024).
 - [35] R. B. Potts, Some generalized order-disorder transformations, *Math. Proc. Camb. Philos. Soc.* **48**, 106–109 (1952).
 - [36] F. Y. Wu, The Potts model, *Rev. Mod. Phys.* **54**, 235 (1982).
 - [37] F. Y. Wu, Potts model and graph theory, *J. Stat. Phys.* **52**, 99 (1988).
 - [38] L. Zdeborová and F. Krzakala, Phase transitions in the coloring of random graphs, *Phys. Rev. E* **76**, 031131 (2007).
 - [39] J. R. McClean, S. Boixo, V. N. Smelyanskiy, R. Babbush, and H. Neven, Barren plateaus in quantum neural network training landscapes, *Nat. Commun.* **9**, 4812 (2018).
 - [40] C. Ortiz Marrero, M. Kieferová, and N. Wiebe, Entanglement-induced barren plateaus, *PRX Quantum*

- [2](#), 040316 (2021).
- [41] D. P. Kingma and J. Ba, Adam: A method for stochastic optimization, *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, edited by Y. Bengio and Y. LeCun (2015).
 - [42] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, *PyTorch: An Imperative Style, High-Performance Deep Learning Library, Proceedings of the 33rd International Conference on Neural Information Processing Systems* (Curran Associates Inc., Red Hook, NY, USA, 2019).
 - [43] W. Li, R. Li, Y. Ma, S. On Chan, D. Pan, and B. Yu, Rethinking graph neural networks for the graph coloring problem, [arXiv:2208.06975](#) (2022).
 - [44] M. Trick, COLOR dataset, URL (accessed Dec. 2023), <https://mat.tepper.cmu.edu/COLOR02/> (2002).
 - [45] A. K. McCallum, K. Nigam, J. Rennie, and K. Seymore, Automating the construction of internet portals with machine learning, *Information Retrieval* **3**, 127 (2000).
 - [46] P. Sen, G. Namata, M. Bilgic, L. Getoor, B. Galligher, and T. Eliassi-Rad, Collective classification in network data, *AI Magazine* **29**, 93 (2008).
 - [47] R. A. Rossi and N. K. Ahmed, The network data repository with interactive graph analytics and visualization, *AAAI* (2015).
 - [48] J. Leskovec and A. Krevl, SNAP Datasets: Stanford large network dataset collection, <http://snap.stanford.edu/data> (2014).
 - [49] A. Hertz and D. de Werra, Using tabu search techniques for graph colorin, *Computing* **39**, 345–351 (1987).
 - [50] A. Kose, B. A. Sonmez, and M. Balaban, Simulated annealing algorithm for graph coloring, [arXiv:1712.00709](#) (2017).
 - [51] A. Nolte and R. Schrader, Simulated annealing and its problems to color graphs, *Algorithms—ESA’96: Fourth Annual European Symposium Barcelona, Spain, September 25–27, 1996 Proceedings 4*, 138–151 (Springer, 1996).
 - [52] S. Łukasik, Z. Kokosiński, and G. Świętoń, Parallel simulated annealing algorithm for graph coloring problem, *Parallel Processing and Applied Mathematics: 7th International Conference, PPAM 2007, Gdansk, Poland, September 9-12, 2007 Revised Selected Papers 7*, 229–238 (Springer, 2008).
 - [53] D. Zheng, C. Ma, M. Wang, J. Zhou, Q. Su, X. Song, Q. Gan, Z. Zhang, and G. Karypis, Distdgl: distributed graph neural network training for billion-scale graphs, *2020 IEEE/ACM 10th Workshop on Irregular Applications: Architectures and Algorithms (IA3)*, 36–44 (IEEE, 2020).
 - [54] W. Hu, M. Fey, M. Zitnik, Y. Dong, H. Ren, B. Liu, M. Catasta, and J. Leskovec, Open graph benchmark: Datasets for machine learning on graphs, *Advances in neural information processing systems* **33**, 22118 (2020).
 - [55] The partially large variances for QdLQA (and possibly for QdGD) comes from the fact that other processes were run simultaneously to the algorithms on the corresponding hardware.
 - [56] <https://github.com/amazon-science/gcp-with-gnns-example>.